Slide 1: 23I501 - Design and Analysis of Algorithms

Batch 15: Cloud Data Centre VM Consolidation

* **Team: 5 Members | Industry: Cloud Computing & Green Data Centres**

* **Objective: Consolidate VMs onto the fewest physical servers to power down idle machines**

* **Key Constraint: Each host server has a fixed capacity of 16 vCPUs**

* **Given Instance: VM vCPU demands [9, 8, 7, 6, 5, 4, 3, 2] -> Min 3 Physical Servers**

* **Techniques Evaluated: Branch & Bound, Greedy (FFD/BFD), Pure Backtracking, D&C, DP**

Slide 2: Problem Context & Industry Motivation

Why Cloud VM Consolidation Matters

* **Hyperscale Cloud Centres: Host tens of thousands of VMs across thousands of physical servers**
* **Idle Power Draw: Physical servers consume 50-60% of peak rated power even with 0% CPU load**
* **Nightly Power-Down Strategy: Consolidate workloads into minimum hosts and power off the rest**
* **Scalability Requirement: Cluster has ~5,000 VMs; placement plan must complete within minutes**
* **Given Batch Instance: 8 VMs with sizes [9, 8, 7, 6, 5, 4, 3, 2] on 16-vCPU hosts**

Slide 3: Problem Modelling & Formal Formulation

Mapping to Classical 1D Bin Packing (NP-Hard)

* **Input: n VMs with vCPU demands s_i in (0, C], server capacity C = 16 vCPUs**
* **Decision Variables: y_j in {0,1} (server j active), x_ij in {0,1} (VM i hosted on server j)**
* **Objective: Minimize sum(y_j) subject to:**
    - 1. Placement Constraint: sum_j(x_ij) = 1 for all VMs i
    - 2. Capacity Constraint: sum_i(s_i * x_ij) <= C * y_j for all servers j
* **Problem Class: Combinatorial Optimisation**
* **NP-Hardness: Proven via polynomial-time reduction from the 3-Partition problem**

Slide 4: Question 1: Theoretical Lower Bounds

Continuous L1 and Martello-Toth L2 Bound

* **Sum of VM demands: 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 = 44 vCPUs**

* **Server Capacity: C = 16 vCPUs**

* **Continuous Lower Bound L1: ceil(sum(s_i) / C) = ceil(44 / 16) = ceil(2.75) = 3 servers**

* **Martello-Toth Lower Bound L2: Evaluates threshold waste partitions -> L2 = 3 servers**

* **Answer to Q1: The fewest number of servers any packing could possibly use is 3 servers**

* **No packing can achieve 2 servers because 2 * 16 = 32 < 44 vCPUs**

Slide 5: Question 4: Data Structure Analysis (5,000 Servers)

Comparing Server Lookup Structures at Hyperscale

* **Scale: Cluster has 5,000 servers and 5,000 VMs**
* **1. Unsorted Array: O(M) search per placement -> O(N*M) = 2.5 * 10^7 ops (Too slow)**
* **2. Max-Segment Tree (Chosen):**
    - Query leftmost server with room >= s_i: O(log M) time
    - Update server residual capacity: O(log M) time
    - Total Cost: O(N log M) = 6.5 * 10^4 ops (< 5 ms for 5,000 VMs) [Optimal for First-Fit]
* **3. Discrete Buckets: Array of linked lists per capacity [0..C] -> O(C) = O(1) [Optimal for Best-Fit]**

Slide 6: Algorithmic Technique: Branch and Bound

Admissible Bounding & Symmetry Breaking

* **Chosen Exact Technique: Depth-First Branch and Bound with Suffix Lower Bounding**

* **Enabling Property 1: Prunable State Space via Admissible Lower Bound**

    o - If current_open_bins + ceil(rem_demand - open_slack)/C >= BestKnown -> PRUNE

* **Enabling Property 2: Canonical Bin Symmetry Breaking**

    o - Only open bin k+1 if bins 1..k are non-empty (eliminates M! permutation explosion)

* **Enabling Property 3: Duplicate Residual Space Pruning**

    o - If two open servers have identical free space, branching into either is symmetric

Slide 7: Rejection of the Other Four Classical Techniques

Concrete Mathematical Proofs & Counterexamples

* **1. Greedy (FFD): Suboptimal. Counterexample: C=16, items [7,6,5,5,5,4] (Sum=32)**
    - Optimal = 2 servers: [7,5,4] and [6,5,5]. FFD = 3 servers: [7,6], [5,5,5], [4]
* **2. Pure Backtracking: Exponential blow-up O(M^N). Visits 1,858 nodes on N=8, >3M on N=14**
* **3. Divide & Conquer: Subproblems not independent; splitting items leaves fragmented bins**
* **4. Dynamic Programming: State space explosion O(C^M) = 16^5000 (catastrophic memory limit)**

Slide 8: Questions 2 & 3: Hand-Trace on Given Instance

Step-by-Step Allocation of [9, 8, 7, 6, 5, 4, 3, 2]

* **VM 9 -> Opens Server 1 [9] (rem: 7 vCPUs)**

* **VM 8 -> Cannot fit in S1 (7 < 8). Opens Server 2 [8] (rem: 8 vCPUs)**

* **VM 7 -> Fits in S1 (7 >= 7). Server 1 [9, 7] -> FULL (rem: 0)**

* **VM 6 -> Fits in S2 (8 >= 6). Server 2 [8, 6] (rem: 2 vCPUs)**

* **VM 5 -> Fits neither S1 nor S2. Opens Server 3 [5] (rem: 11 vCPUs)**

* **VM 4 & VM 3 -> Fit in S3. Server 3 [5, 4, 3] (rem: 4 vCPUs)**

* **VM 2 -> Fits in S2 (2 >= 2). Server 2 [8, 6, 2] -> FULL (rem: 0)**

* **Final Optimal Allocation: Exactly 3 Physical Servers (11 search nodes explored in B&B!)**

Slide 9: Formal Proof of Correctness

Inductive Invariant of Bounding Function

* **Theorem: B&B with L1 suffix bounding never discards any optimal packing**

* **Proof:**

    - 1. Let state sigma have k open bins and remaining unplaced VMs U with total demand sum(U)

    - 2. Total available slack across open bins is slack_open

    - 3. Even with zero packing waste, excess demand max(0, sum(U) - slack_open) requires

    - at least Delta = ceil(excess / C) new servers

    - 4. Any complete solution extending sigma requires at least k + Delta servers

    - 5. If k + Delta >= BestKnown, no leaf in subtree can beat BestKnown -> Pruning is safe Q.E.D.

Slide 10: Empirical Benchmarking & Timing Analysis

Empirical Verification across 6 Input Sizes

* **Benchmark Setup: Tested on sizes N in [6, 8, 10, 12, 14, 16], averaged over 3 trials**

* **N=6: FFD: 0.033 ms | B&B: 0.086 ms (7 nodes) | Pure BT: 0.268 ms (240 nodes)**

* **N=8: FFD: 0.042 ms | B&B: 0.160 ms (10 nodes) | Pure BT: 2.141 ms (1,979 nodes)**

* **N=10: FFD: 0.045 ms | B&B: 0.097 ms (13 nodes) | Pure BT: 37.400 ms (46,689 nodes)**

* **N=12: FFD: 0.068 ms | B&B: 0.146 ms (16 nodes) | Pure BT: 499.017 ms (515,953 nodes)**

* **N=14: FFD: 0.130 ms | B&B: 0.186 ms (20 nodes) | Pure BT: 3,000 ms TIMEOUT (>3M nodes)**

* **Conclusion: Branch & Bound achieves >99.999% pruning efficiency compared to Pure Backtracking**

Slide 11: Worst-Case Input Construction

Adversarial Instances for FFD / B&B

* **Adversarial Construction for FFD (Johnson & Garey):**
    - o - Construct 6k items grouped as: 2k of (1/2 + eps), 2k of (1/4 + 2*eps), 2k of (1/4 + eps)
    - o - Optimal packing combines one from each group: (1/2+eps) + (1/4+2eps) + (1/4+eps) = 1 -> 2k bins
    - o - FFD packs identical large items together, stranding capacity: uses ~ (11/9)*2k = (22/9)k bins
* **Matches the theoretical tight approximation ratio: FFD(I) <= (11/9)*OPT(I) + 6/9**

Slide 12: Question 5: Changed Requirement

2D Vector Bin Packing (vCPU + RAM) & Anti-Affinity

* **Added Constraints:**
    - o 1. 2D Resources: Each VM requires (c_i vCPUs, r_i RAM); Server has (16 vCPU, 64 GB RAM)
    - o 2. Anti-Affinity: Two replicas of the same service cannot share a server (Fault Domain)
* **Revised Theoretical Lower Bound:**
    - o L_revised = max(ceil(sum(c_i)/16), ceil(sum(r_i)/64), max_service |replicas|)
    - o - The third term is the Clique Number omega(G) of the microservice conflict graph
* **Revised Algorithm: Multidimensional FFD with Anti-Affinity (MFFD-AA) using server hash-sets**

Slide 13: Real-World Industry Reference

Kubernetes kube-scheduler & VMware vSphere DRS

* **Real Cloud Systems: Kubernetes kube-scheduler & VMware vSphere Distributed Resource Scheduler**

* **Two-Phase Scheduling Pipeline:**

    o 1. Filtering Phase (Predicates): NodeResourcesFit (CPU, RAM) & podAntiAffinity rules

    o 2. Scoring Phase (Priorities): NodeResourcesBalancedAllocation (avoids resource starvation)

* **Key Differences from Our Model:**

    o - Offline vs Online: Our model solves static batch; Kubernetes solves streaming arrival/departure

    o - Live Migration Overhead: Real DRS considers VM network latency and memory transfer churn

Slide 14: Conclusion & Team Summary

23I501 DAA Assignment Submission

* **All 8 Rubric Components C1-C8 Covered with Formal Mathematical Proofs**
* **Member 1: Problem Modelling & Theoretical Lower Bounds (Q1)**
* **Member 2: Segment Tree Data Structure Analysis for 5,000 Servers (Q4)**
* **Member 3: Branch and Bound Algorithm & Concrete 4-Technique Rejection**
* **Member 4: Execution Trace, Correctness Proof & Empirical Benchmarks**
* **Member 5: Changed Requirement (2D + Anti-Affinity) & Real-World System (Q5)**
* **Thank you! We are now open for Viva Voce questions.**